

GUIDA PER IL FORMATORE

Capitolo 2: Claude for Code

Corso di Formazione su Claude Desktop e Claude for Code



Scopo del Documento

Questa guida fornisce al formatore tutti gli elementi necessari per condurre efficacemente le lezioni del Capitolo 2, dedicato a Claude for Code. Include fondamenti teorici, esempi pratici multipli, esercizi hands-on completi, casi d'uso reali, e note pedagogiche dettagliate.



Target e Approccio Pedagogico

Target: gruppo eterogeneo con livelli di competenza variabili (da principianti a intermedi).
Approccio: 30% teoria / 70% pratica, con enfasi su applicazioni concrete e spec-driven development. Ogni concetto è supportato da 3-5 esempi pratici diversificati.

Indice dei Contenuti

GUIDA PER IL FORMATORE.....	1
Indice dei Contenuti.....	2
2.1 Installazione, Setup e Configurazione	4
1. Introduzione al Topic	4
Obiettivi di Apprendimento	4
Prerequisiti	4
Rilevanza nel Lavoro Quotidiano	5
2. Fondamenti Teorici	6
Architettura di Claude for Code	6
Modelli di Installazione	6
Sistema di Autenticazione	7
Configurazione e File di Setup	7
Modalità Operative.....	8
1. Modalità Interattiva	8
2. Modalità Batch	8
3. Modalità CI/CD	8
3. Applicazioni Pratiche	9
A. Dimostrazione Passo-Passo: Installazione Completa.....	9
B. Esempi Pratici Multipli.....	10
4. Casi d'Uso Reali	12
5. Best Practices e Linee Guida.....	13
Checklist Pre-Installazione	13
Best Practices di Configurazione	13
Cosa Fare e Cosa Evitare	14
7. Risorse Supplementari	15
Documentazione Ufficiale	15
Strumenti e Utility	15
Community e Support	15
Lecture Consigliate per Approfondimento	15
2.2 Interfaccia a Linea di Comando e Workflow	16
1. Introduzione al Topic	16
Obiettivi di Apprendimento	16
2. Fondamenti Teorici	16
Anatomia di un Comando Claude	16
Pattern di Workflow	17

3. APPLICAZIONI PRATICHE	18
Demo 1: Sessione Interattiva Completa (25 minuti)	18
4. ESERCIZI HANDS-ON	18
Esercizio 2: Workflow Automation (30 minuti)	18
5. CASI D'USO REALI	19
Caso 1: Startup - Rapid Prototyping.....	19
Caso 2: Enterprise - Legacy Modernization	19
6. BEST PRACTICES E LINEE GUIDA.....	20
7. TROUBLESHOOTING.....	21
Problema 1: Output Non Atteso	21
Problema 2: Performance Lente	21
Problema 3: Workflow Inefficiente.....	21

2.1 Installazione, Setup e Configurazione

1. Introduzione al Topic

Claude for Code rappresenta un'evoluzione significativa nell'integrazione di Claude con i workflow di sviluppo software. Mentre Claude Desktop offre un'interfaccia web potente e versatile, Claude for Code porta le capacità dell'AI direttamente nel terminal, l'ambiente nativo degli sviluppatori. Questo strumento da linea di comando permette di delegare task di coding complessi, automatizzare operazioni ripetitive, e integrare Claude in pipeline di sviluppo esistenti.

Obiettivi di Apprendimento

 **Al termine di questo modulo, i partecipanti saranno in grado di:**

- Installare Claude for Code su qualsiasi sistema operativo utilizzando il metodo più appropriato
- Configurare correttamente l'ambiente di sviluppo e le credenziali di autenticazione
- Personalizzare le impostazioni per diversi contesti di lavoro e progetti
- Integrare Claude for Code con strumenti esistenti (IDE, git, build systems)
- Risolvere problemi comuni di installazione e configurazione
- Ottimizzare il setup per massimizzare produttività e workflow

Prerequisiti

Conoscenze richieste:

- Familiarità di base con l'utilizzo del terminal/prompt dei comandi
- Comprensione dei concetti di variabili d'ambiente
- Conoscenza base di gestione pacchetti (npm, pip, o equivalenti)
- Account Claude attivo con credenziali di accesso

Requisiti tecnici:

- Sistema operativo: Windows 10+, macOS 10.15+, o Linux (distribuzione moderna)
- Node.js versione 16 o superiore installato
- Almeno 500MB di spazio disco disponibile
- Connessione internet stabile per il download e l'autenticazione

Rilevanza nel Lavoro Quotidiano

Claude for Code trasforma il modo in cui gli sviluppatori interagiscono con l'AI durante il coding. Invece di copiare codice avanti e indietro tra browser e IDE, Claude for Code permette di mantenere il flusso di lavoro nel terminal. Questo si traduce in:

- Maggiore velocità: nessun context switching tra applicazioni diverse
- Workflow più naturale: il terminal è l'ambiente nativo per gli sviluppatori
- Automazione avanzata: possibilità di scriptare interazioni con Claude
- Integrazione seamless: Claude diventa parte della toolchain di sviluppo
- Controllo granulare: precisione maggiore nelle operazioni rispetto all'interfaccia web

2. Fondamenti Teorici

Architettura di Claude for Code

Claude for Code è strutturato come un'applicazione client che comunica con i server di Claude tramite API sicure. L'architettura si compone di tre livelli principali:

1. Livello CLI (Command Line Interface): il punto di ingresso per l'utente, gestisce parsing dei comandi, validazione input, e presentazione output
2. Livello di Comunicazione: gestisce l'autenticazione, la comunicazione con le API di Claude, e la gestione dello stato della sessione
3. Livello di Gestione Contesto: mantiene il contesto locale (file, configurazioni, storia delle interazioni) e lo sincronizza con le richieste all'API

Differenza Chiave con Claude Desktop

Mentre Claude Desktop mantiene tutto il contesto sul server (conversazioni, progetti, file caricati), Claude for Code opera con un modello ibrido: il contesto locale rimane sulla macchina dell'utente, e solo le richieste specifiche vengono inviate al server. Questo offre maggiore controllo ma richiede gestione esplicita del contesto.

Modelli di Installazione

Esistono diversi approcci all'installazione di Claude for Code, ciascuno con vantaggi e svantaggi specifici:

A. Installazione tramite npm (Node Package Manager)

- Vantaggi: installazione rapida, aggiornamenti semplici, compatibilità multiplatforma
- Svantaggi: richiede Node.js preinstallato, dimensioni package relativamente grandi
- Ideale per: sviluppatori JavaScript/TypeScript, ambienti con Node.js già presente

B. Installazione tramite gestori di pacchetti OS-specifici

- Windows: via Chocolatey o Scoop
- macOS: via Homebrew
- Linux: via snap, apt, yum secondo la distribuzione
- Vantaggi: integrazione nativa con il sistema, gestione dipendenze automatica
- Ideale per: amministratori di sistema, deployment enterprise

Sistema di Autenticazione

Claude for Code utilizza un sistema di autenticazione basato su token API. Il processo funziona come segue:

4. L'utente genera un API key dal dashboard di Claude (claude.ai/settings)
5. Il token viene configurato localmente tramite variabile d'ambiente o file di configurazione
6. Ad ogni richiesta, Claude for Code include il token nell'header HTTP
7. Il server valida il token e autorizza la richiesta

Sicurezza dei Token

I token API sono credenziali sensibili equivalenti a password. Non devono mai essere committati in repository git, condivisi pubblicamente, o inclusi in codice sorgente. Utilizzare sempre variabili d'ambiente o file di configurazione con permessi restrittivi.

Configurazione e File di Setup

Claude for Code utilizza una gerarchia di configurazioni che permette flessibilità e override granulare:

8. Configurazione Globale: applicata a tutti i progetti (`~/.claude/config`)
9. Configurazione di Progetto: specifica per directory (`.claude/config`)
10. Variabili d'Ambiente: override temporaneo (es. `CLAUDE_API_KEY`)
11. Parametri CLI: override per singolo comando (`--model`, `--temperature`, etc.)

La priorità è: Parametri CLI > Variabili d'Ambiente > Config Progetto > Config Globale

Modalità Operative

Claude for Code supporta tre modalità operative principali, ciascuna ottimizzata per contesti di lavoro specifici.

1. Modalità Interattiva

La modalità interattiva è la configurazione predefinita, ideale per sviluppo esplorativo e debugging.

 **Quando Usare** • Sviluppo esplorativo e prototipazione rapida • Debugging di problemi complessi • Refactoring con decisioni contestuali • Apprendimento di nuovi framework • Pair programming con Claude

2. Modalità Batch

Esecuzione automatizzata di task predefiniti senza interazione umana.

 **Quando Usare** • Generazione automatica da specifiche complete • Refactoring standard su codebase esteso • Generazione test automatizzata • Documentazione automatica API • Migration tra versioni framework

3. Modalità CI/CD

Integrazione in pipeline di continuous integration e deployment con focus su sicurezza e automazione.

 **Quando Usare** • Code review automatizzato su PR • Generazione release notes da commits • Validazione contro specifiche architetturali • Security scanning potenziato • Performance analysis automatizzato

3. Applicazioni Pratiche

A. Dimostrazione Passo-Passo: Installazione Completa

Questa sezione fornisce una guida dettagliata per l'installazione completa di Claude for Code su diversi sistemi operativi.

Scenario 2: Installazione su Windows via npm

Passo 1: Verifica prerequisiti

Aprire PowerShell o Command Prompt e verificare Node.js:



Comando

```
node --version npm --version
```

Passo 2: Installazione globale



Comando

```
npm install -g @anthropic-ai/claude-code
```



Nota per Windows

Su Windows, potrebbe essere necessario eseguire PowerShell come Amministratore per l'installazione globale. Se si riceve un errore di permessi, chiudere PowerShell, riaprirlo come amministratore (tasto destro > Esegui come amministratore), e riprovare.

Passo 3: Configurazione API key su Windows

Su Windows, configurare la variabile d'ambiente permanentemente tramite System Properties:

12. Aprire 'Modifica variabili d'ambiente di sistema' dal menu Start
13. Cliccare 'Variabili d'ambiente'
14. Nella sezione 'Variabili utente', cliccare 'Nuova'
15. Nome variabile: CLAUDE_API_KEY
16. Valore variabile: la propria API key (sk-ant-api03-...)
17. Cliccare OK per salvare
18. Chiudere e riaprire PowerShell per applicare le modifiche

Alternativa via PowerShell (temporanea per la sessione corrente):



Comando

```
$env:CLAUDE_API_KEY = 'sk-ant-api03-...'
```

B. Esempi Pratici Multipli

Questa sezione presenta 5 esempi pratici diversificati che coprono diversi scenari di configurazione e personalizzazione di Claude for Code.

Esempio 1: Configurazione Multi-Progetto (Livello Principiante)

Scenario: uno sviluppatore lavora su tre progetti diversi, ciascuno con requisiti di configurazione differenti.

Progetto A (startup-web): preferenza per output conciso

File .claude/config nel progetto A

```
{ "model": "claude-sonnet-4-5", "temperature": 0.7, "max_tokens": 2000, "style": "concise" }
```

Progetto B (enterprise-backend): output dettagliato con focus su sicurezza

File .claude/config nel progetto B

```
{ "model": "claude-opus-4-5", "temperature": 0.3, "max_tokens": 4000, "style": "detailed", "system_prompt": "Focus on security best practices and detailed error handling" }
```

Progetto C (data-science): configurazione per analisi dati

File .claude/config nel progetto C

```
{ "model": "claude-sonnet-4-5", "temperature": 0.5, "max_tokens": 3000, "context_files": ["/data/schema.json", "/docs/analysis-guide.md"] }
```

Vantaggi di Questo Approccio

Ogni progetto mantiene la propria configurazione ottimizzata. Cambiando directory, Claude for Code applica automaticamente la configurazione appropriata senza intervento manuale. Questo è particolarmente utile quando si lavora su progetti paralleli con requisiti diversi.

Esempio 5: Troubleshooting di Problemi Comuni (Livello Principiante)

Scenario: diagnosticare e risolvere i problemi di configurazione più frequenti.

Problema 1: 'Comando claude non trovato'

Diagnosi e Soluzione

Causa: npm global bin directory non nel PATH Verifica: `echo $PATH | grep npm` npm config get prefix Soluzione: `export PATH="$(npm config get prefix)/bin:$PATH"` echo 'export PATH="\$(npm config get prefix)/bin:\$PATH"' >> ~/.bashrc

Problema 2: 'Authentication failed'

Diagnosi e Soluzione

Causa: API key non configurata o invalida Verifica: `echo $CLAUDE_API_KEY` claude --debug auth-test Soluzione: 1. Verificare API key su claude.ai/settings/api-keys 2. Rigenerare se necessario 3. Riconfigurare: `export CLAUDE_API_KEY='nuova-key'` echo 'export CLAUDE_API_KEY="nuova-key"' >> ~/.bashrc

Problema 3: Performance lente o timeout

Diagnosi e Soluzione

Cause possibili: connessione lenta, file troppo grandi, configurazione suboptimale Verifica: `claude --debug network-test` claude config show Soluzioni: • Ridurre max_tokens in config • Usare --file invece di stdin per file grandi • Abilitare caching: `claude config set cache.enabled true` • Verificare connessione: `ping claude.ai`

4. Casi d'Uso Reali

Questa sezione esplora scenari reali in cui una configurazione appropriata di Claude for Code porta vantaggi concreti nel lavoro quotidiano.

Caso d'Uso 1: Team di Sviluppo Distribuito

Contesto:

Un team di 15 sviluppatori distribuiti in 3 fusi orari lavora su una applicazione enterprise. Servono standard condivisi ma flessibilità per preferenze individuali.

Soluzione Implementata:

- Configurazione team condivisa nel repository (team-claude-config.json)
- Script onboarding automatizzato per nuovi membri
- Config locali per preferenze individuali (override di team config)
- Dashboard centralizzato per monitoraggio utilizzo e costi
- Template pre-configurati per task comuni (review, testing, documentation)

Risultati Misurabili:

- Tempo di onboarding ridotto da 1 giorno a 30 minuti
- Consistenza nelle code review aumentata del 40%
- Riduzione del 25% nei bug identificati post-merge
- ROI positivo entro 2 settimane dall'adozione

Caso d'Uso 2: Sviluppatore Freelance Multi-Cliente

Contesto:

Uno sviluppatore freelance gestisce 5 progetti per clienti diversi, ciascuno con stack tecnologico, coding standards, e requisiti di documentazione differenti.

Soluzione Implementata:

- Configurazione specifica per ogni cliente nella directory del progetto
- Alias personalizzati per workflow ricorrenti per cliente
- Template di prompt specifici per standard di codifica di ciascun cliente
- System prompts customizzati che incorporano linee guida del cliente
- Tracking separato di utilizzo per fatturazione al cliente

Risultati Misurabili:

- Zero errori di cross-contamination tra standard di clienti diversi
- Velocità di context switching tra progetti aumentata del 60%
- Soddisfazione cliente migliorata per consistenza output
- Capacità di gestire +2 progetti contemporaneamente

5. Best Practices e Linee Guida

Checklist Pre-Installazione

✓ Verifiche da Completare Prima di Installare

- ☐ Sistema operativo supportato e aggiornato
- ☐ Spazio disco sufficiente (almeno 500MB liberi)
- ☐ Node.js versione 16+ installato e funzionante
- ☐ Accesso internet stabile (almeno 2 Mbps)
- ☐ Account Claude attivo con API key generata
- ☐ Permessi di scrittura su directory npm global (o configurato npm prefix custom)
- ☐ Terminal/shell preferita configurata correttamente
- ☐ Git installato se si prevede integrazione con hook

Best Practices di Configurazione

1. Gestione Sicura delle Credenziali

- NON committare mai API keys in repository git
- Usare variabili d'ambiente invece di hardcoding in file
- Aggiungere *.key, .env, *credentials* a .gitignore
- Ruotare API keys periodicamente (ogni 3-6 mesi)
- Usare API keys separate per ambienti diversi (dev, staging, prod)

2. Organizzazione Configurazioni

- Usare config globale solo per setting davvero universali
- Preferire config a livello progetto per massima flessibilità
- Documentare configurazioni custom con commenti inline
- Versionare config di progetto (sono parte del codice)
- Creare template riutilizzabili per tipi di progetto comuni

3. Ottimizzazione Performance

- Impostare max_tokens appropriati (non sempre il massimo è meglio)
- Usare temperatura bassa (0.2-0.4) per task deterministici
- Abilitare caching per context ripetuti
- Preferire --file a stdin per file grandi (>100KB)
- Considerare modelli più veloci (Haiku) per task semplici

4. Manutenzione e Aggiornamenti

- Controllare aggiornamenti mensilmente: `npm update -g @anthropic-ai/claude-code`
- Leggere changelog prima di aggiornare versioni major
- Testare su progetto non-critico prima di upgrade in produzione
- Mantenere backup delle configurazioni funzionanti
- Documentare modifiche custom per future reference

Cosa Fare e Cosa Evitare

☑ DA FARE:

- Iniziare con configurazione semplice e iterare
- Testare configurazione dopo ogni modifica
- Documentare decisioni di configurazione per team
- Usare naming consistente per file di config
- Creare alias shell per comandi frequenti
- Sperimentare con diversi modelli per trovare il migliore per uso specifico

✗ DA EVITARE:

- Configurare tutto in una sessione (approccio incrementale è migliore)
- Copiare configurazioni da internet senza capirle
- Ignorare warning e errori durante setup
- Usare sudo con npm install (configurare npm prefix invece)
- Condividere API key tra membri del team
- Modificare file di sistema senza backup

7. Risorse Supplementari

Documentazione Ufficiale

- Claude for Code GitHub Repository: <https://github.com/anthropics/claude-code>
- Documentazione API: <https://docs.anthropic.com/claude/reference>
- Installation Guide: <https://docs.anthropic.com/claude/docs/cli-installation>
- Configuration Reference: <https://docs.anthropic.com/claude/docs/cli-config>

Strumenti e Utility

- Node.js: <https://nodejs.org/>
- Visual Studio Code: <https://code.visualstudio.com/>
- Notepad++: <https://notepad-plus-plus.org/>
- GitHub Desktop: <https://desktop.github.com/>

Community e Support

- Claude Discord Community: discord.gg/claude
- Stack Overflow: tag [\[claude-ai\]](#)
- Anthropic Support: support@anthropic.com

Lectture Consigliate per Approfondimento

- 'The Pragmatic Programmer' - per contesto su developer tools e CLI
- 'Working Effectively with Legacy Code' - per pattern di refactoring assistito da AI
- Blog Anthropic: anthropic.com/blog - per aggiornamenti e nuove feature

2.2 Interfaccia a Linea di Comando e Workflow

1. Introduzione al Topic

L'interfaccia a linea di comando di Claude for Code rappresenta il cuore operativo dello strumento. Dopo aver completato installazione e configurazione, è ora di imparare come utilizzare effettivamente Claude dal terminal per risolvere problemi reali di sviluppo. Questo modulo trasforma la configurazione teorica in capacità operative concrete.

La CLI di Claude for Code è progettata per integrarsi naturalmente nel workflow degli sviluppatori. Non si tratta di un add-on separato che richiede context switching, ma di uno strumento che diventa parte organica del flusso di lavoro quotidiano nel terminal. Comprendere i comandi disponibili, come comporli, e come orchestrarli in workflow efficaci è essenziale per massimizzare la produttività.

Obiettivi di Apprendimento

 **Al termine di questo modulo, i partecipanti saranno in grado di:**

- Utilizzare tutti i comandi principali della CLI di Claude for Code
- Comporre comandi per workflow complessi
- Integrare Claude for Code con altri strumenti da linea di comando
- Orchestrare sessioni di lavoro efficienti per diversi tipi di task
- Automatizzare operazioni ripetitive tramite script
- Applicare pattern di workflow per scenari comuni di sviluppo

2. Fondamenti Teorici

Anatomia di un Comando Claude

Ogni comando Claude segue una struttura standard che comprende il comando base, azioni opzionali, parametri, e flag di configurazione:

Struttura Base

`claude [action] [input] [flags]` Esempio: `claude review src/main.py --model opus --output review.md`

Componenti:

- **claude**: il comando base, punto di ingresso
- **action**: operazione da eseguire (review, generate, explain, debug, etc.)
- **input**: il target dell'azione (file, directory, testo)
- **flags**: modificatori che alterano comportamento (`--model`, `--temperature`, `--output`)

COMANDI CORE:

- claude generate - Genera nuovo codice da specifiche
- claude review - Analizza codice per bug e code smells
- claude refactor - Ristruttura codice mantenendo funzionalità
- claude test - Genera test cases per codice esistente
- claude debug - Analizza errori e suggerisce fix
- claude explain - Spiega codice complesso
- claude document - Genera documentazione

Pattern di Workflow

I workflow tipici seguono pattern ricorrenti che ottimizzano produttività e qualità:



Pattern: Feature Development 1. Genera implementazione da spec 2. Review automatico per quality 3. Applica suggerimenti di refactoring 4. Genera test coverage 5. Documenta il codice 6. Integra in codebase esistente



Pattern: Legacy Modernization 1. Analizza architettura esistente 2. Identifica bounded contexts 3. Estrai un servizio alla volta 4. Genera anti-corruption layers 5. Crea integration tests 6. Valida e deploia

3. APPLICAZIONI PRATICHE

Demo 1: Sessione Interattiva Completa (25 minuti)

Scenario: Sviluppare una REST API per gestione tasks con CRUD completo

 **Step-by-Step Workflow** STEP 1 - Inizializzazione (5 min): `claude init --language python --framework flask` STEP 2 - Generazione Struttura (8 min): Prompt interattivo per models, endpoints, validazione STEP 3 - Review e Iterazione (7 min): Feedback incrementali per aggiungere features STEP 4 - Test Generation (5 min): `claude test api/ --coverage 85`

4. ESERCIZI HANDS-ON

Esercizio 2: Workflow Automation (30 minuti)

TASK: Creare script bash per automated feature development completo

 **Lo Script Deve** 1. Accettare feature spec come input 2. Generare implementazione 3. Eseguire code review automatico 4. Applicare refactoring suggerito 5. Generare test suite 6. Validare coverage 7. Generare documentazione 8. Creare PR con contesto completo

5. CASI D'USO REALI

Caso 1: Startup - Rapid Prototyping

CONTESTO: Startup in fase seed, 3 developers, MVP in 2 settimane

WORKFLOW IMPLEMENTATO:

- Morning: Product owner scrive user stories → Claude genera spec tecniche
- Development: Claude genera boilerplate → Developers customizzano business logic
- Evening: Claude review automatico → Documentazione updated → Deploy

RISULTATI:

- MVP delivered in 11 giorni (vs 14)
- Code quality 8.5/10 (SonarQube)
- Test coverage 87%
- Technical debt controllato

Caso 2: Enterprise - Legacy Modernization

CONTESTO: Modernizzare monolite Java EE (500K lines) a microservices

APPROCCIO CON CLAUDE:

- FASE 1: Analysis - Claude analizza dependencies e identifica bounded contexts
- FASE 2: Extraction - Estrae un bounded context alla volta con API gateway
- FASE 3: Testing - Genera integration tests e contract testing automatizzati

METRICHE:

- 16 microservices estratti in 8 settimane
- 60% reduction in time vs manual
- Zero critical bugs post-migration
- Team velocity +40%

6. BEST PRACTICES E LINEE GUIDA

 **Session Planning** 1. DEFINIRE obiettivo chiaro 2. PREPARARE context necessario 3. INIZIARE con high-level approach 4. ITERARE con feedback incrementali 5. VALIDARE prima di procedere 6. DOCUMENTARE decisioni chiave

OTTIMIZZAZIONE PERFORMANCE:

- Usare --cache per context ripetuti
- Preferire batch mode per > 5 file simili
- Utilizzare --fast flag con Haiku per task semplici
- Parallelizzare richieste indipendenti

PATTERN ANTI-PATTERN:

DO:

- Fornire context chiaro e spec dettagliate
- Iterare con feedback specifici
- Validare output prima di applicare
- Usare version control per ogni modifica

DON'T:

- Dare istruzioni vaghe
- Accettare prima versione senza review
- Applicare modifiche massive senza verifica
- Modificare direttamente produzione

7. TROUBLESHOOTING

Problema 1: Output Non Atteso

 **Diagnosi e Soluzione** ROOT CAUSE: Prompt ambiguo o context insufficiente VERIFICHE: • Il prompt è chiaro e specifico? • Ci sono esempi concreti? • Il context include tutte le info necessarie? SOLUZIONI: • Riscrivere prompt con maggior dettaglio • Fornire esempi di input/output desiderato • Includere file di riferimento rilevanti • Usare --verbose per debug del reasoning

Problema 2: Performance Lente

 **Soluzioni** 1. Split file grandi in chunks 2. Process incrementalmente 3. Usa modello più veloce (Haiku) 4. Abilita caching per context ripetuti

Problema 3: Workflow Inefficiente

 **Checklist Ottimizzazione** ☐ Sto usando batch mode dove appropriato? ☐ Ho configurato alias per comandi frequenti? ☐ Sto usando il modello giusto per il task? ☐ Ho abilitato caching? ☐ Sto parallelizzando operazioni indipendenti? ☐ Ho automatizzato workflow ripetitivi?